

Interprocess Communication

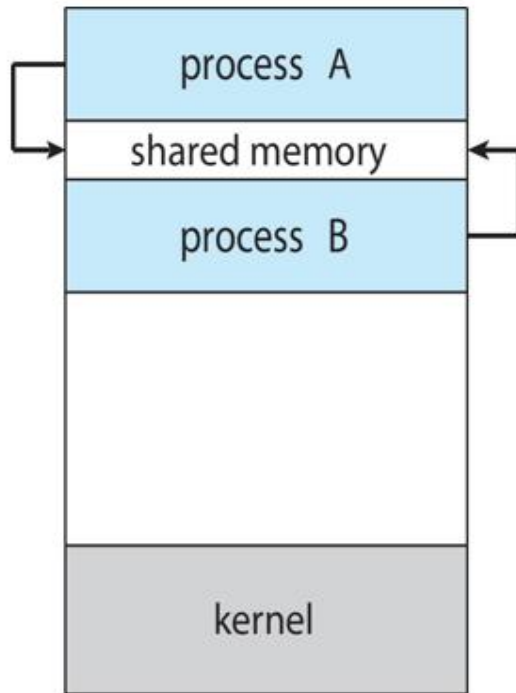
- Processes within a system may be *independent* or *cooperating*
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
 - Convenience
- Cooperating processes need **interprocess communication (IPC)**
- Two models of IPC
 - **Shared memory**
 - **Message passing**



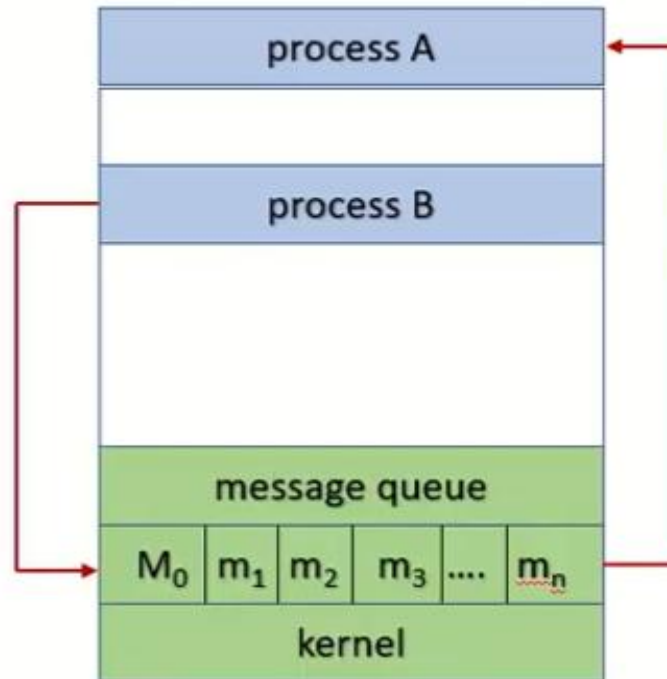


Communications Models

(a) Shared memory.



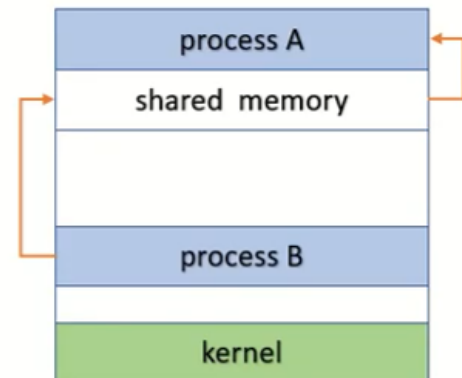
(b) Message passing.





IPC – Shared Memory

- Area of shared memory in address space of process creating shared-memory segment
- Other processes must attach it to their address space
- Both processes can then use it like regular memory
- Can exchange information by reading, writing data
- Communication is under control of users processes
- Advantage - Fast
- Limitation - To provide mechanism for user processes to synchronize when they access shared memory





IPC – Shared Memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.
- Synchronization is discussed in great details in Chapters 6 & 7.

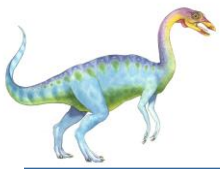




Producer – Consumer Problem

- Paradigm for cooperating processes:
 - *producer* process produces information that is consumed by a *consumer* process
- Two variations:
 - **unbounded-buffer** places no practical limit on the size of the buffer:
 - ▶ Producer never waits
 - ▶ Consumer waits if there is no buffer to consume
 - **bounded-buffer** assumes that there is a fixed buffer size
 - ▶ Producer must wait if all buffers are full
 - ▶ Consumer waits if there is no buffer to consume





Bounded-Buffer – Shared-Memory Solution

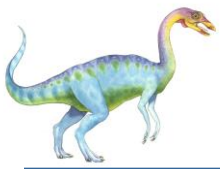
- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Solution is correct, but can only use **BUFFER_SIZE-1** elements





Producer Process – Shared Memory

```
item next_produced;

while (true) {
    /* produce an item in next produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```





Consumer Process – Shared Memory

```
item next_consumed;

while (true) {
    while (in == out)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next consumed */
}
```





What about Filling all the Buffers?

- Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers.
- We can do so by having an integer **counter** that keeps track of the number of full buffers.
- Initially, **counter** is set to 0.
- The integer **counter** is incremented by the producer after it produces a new buffer.
- The integer **counter** is and is decremented by the consumer after it consumes a buffer.

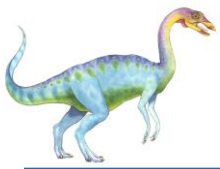




Producer

```
while (true) {  
    /* produce an item in next produced */  
  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```





Consumer

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in next consumed */  
}
```





Examples of IPC Systems - POSIX

■ POSIX Shared Memory

- Process first creates shared memory segment

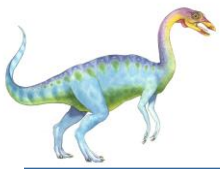
```
shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
```

- Also used to open an existing segment
- Set the size of the object

```
ftruncate(shm_fd, 4096);
```

- Use `mmap()` to memory-map a file pointer to the shared memory object
- Reading and writing to shared memory is done by using the pointer returned by `mmap()`.





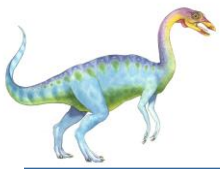
Examples of IPC Systems - POSIX

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

```
ptr = mmap(0, 4096, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
```

```
mmap(  
    address,    // Starting address where the mapping should  
                // be placed. Usually 0 or NULL  
    size,       // Number of bytes  
    protection, // Read/Write permissions  
    sharing,    // Shared or private  
    file descriptor, // Returned by shm_open()  
    offset      // Starting position in the file/shared memory  
                // (usually 0).  
);
```

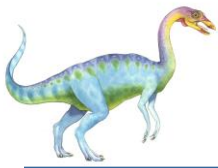




Examples of IPC Systems - POSIX

```
int shm_fd;  
char *ptr;  
  
shm_fd = shm_open("/OS", O_CREAT | O_RDWR, 0666);  
ftruncate(shm_fd, 4096);  
  
ptr = mmap(0, 4096, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);  
  
/* Use the shared memory */  
  
/* Remove the mapping */  
munmap(ptr, 4096);  
  
/* Close the file descriptor */  
close(shm_fd);  
  
/* Delete the shared memory object */  
shm_unlink("/OS");
```





IPC POSIX Producer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* strings written to shared memory */
    const char *message_0 = "Hello";
    const char *message_1 = "World!";

    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void *ptr;

    /* create the shared memory object */
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

    /* configure the size of the shared memory object */
    ftruncate(shm_fd, SIZE);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    /* write to the shared memory object */
    sprintf(ptr, "%s", message_0);
    ptr += strlen(message_0);
    sprintf(ptr, "%s", message_1);
    ptr += strlen(message_1);

    return 0;
}
```





IPC POSIX Consumer

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void *ptr;

    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    /* read from the shared memory object */
    printf("%s", (char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

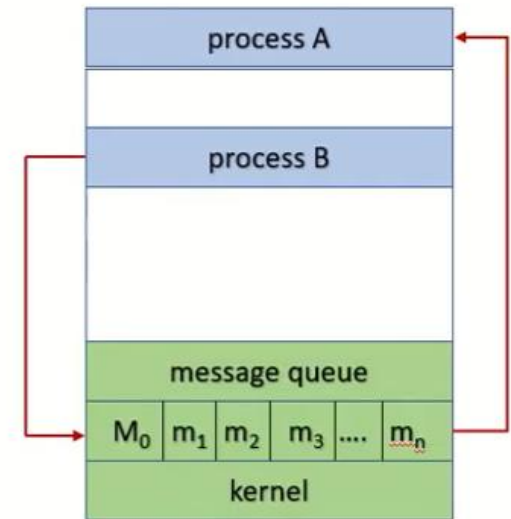
    return 0;
}
```





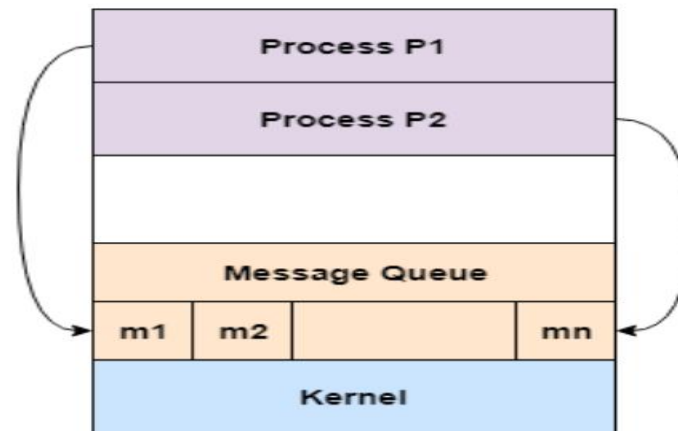
IPC – Message Passing

- Shared memory created in kernel
- System calls used for communicating:
 - **send** (*A, message*) – send a message to process A
 - **receive**(*B, message*) – receive a message from process B
- Advantage: Explicit sharing, less error prone
- Limitation: slow



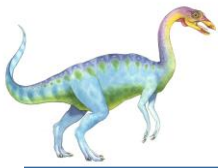


- Message passing model allows multiple processes to read and write data to the message queue without being connected to each other.
- Messages are stored on the queue until their recipient retrieves them. Message queues are quite useful for interprocess communication and are used by most operating systems.



Message Passing Model

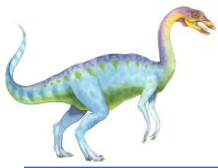




Direct Communication

- Processes must name each other explicitly:
 - **send** (P , *message*) – send a message to process P
 - **receive**(Q , *message*) – receive a message from process Q
- Properties of communication link
 - Links are established automatically
 - A link is associated with exactly one pair of communicating processes
 - Between each pair there exists exactly one link
 - The link may be unidirectional, but is usually bi-directional





Indirect Communication

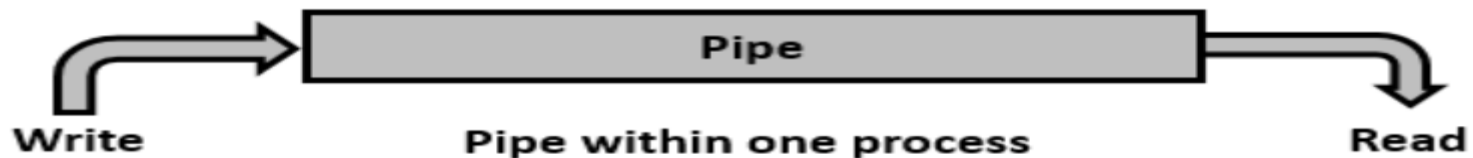
- Messages are directed and received from mailboxes (also referred to as ports)
 - Each mailbox has a unique id
 - Processes can communicate only if they share a mailbox
- Properties of communication link
 - Link established only if processes share a common mailbox
 - A link may be associated with many processes
 - Each pair of processes may share several communication links
 - Link may be unidirectional or bi-directional





Pipes

- Pipe is a communication medium between two or more related or interrelated processes.
- It can be either within one process or a communication between the child and the parent processes.
- Communication is achieved by one process writing into the pipe and other reading from the pipe.
- It is created using **the pipe() system** call and is identified by a file descriptor





Ordinary Pipes

- Acts as a conduit allowing two processes to communicate
- Typically, parent process creates pipe to communicate with a child process
- Since a pipe is a special type of file, child inherits pipe from its parent process
- Ordinary pipes constructed using function

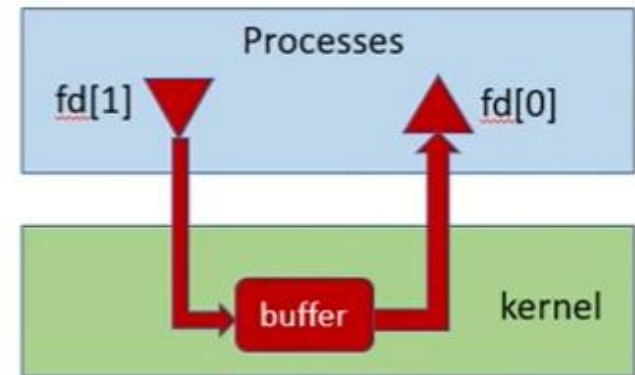
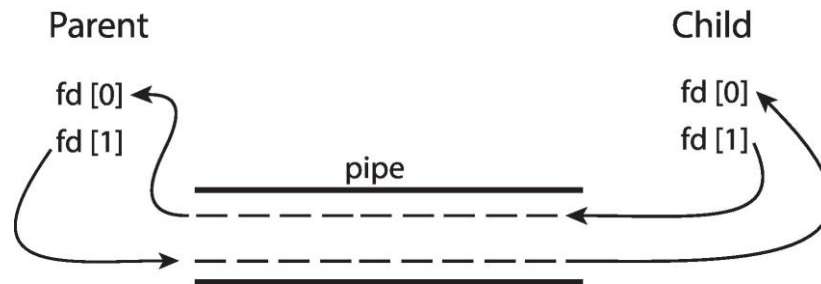
```
pipe(int fd[])
```





Ordinary Pipes

- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Ordinary pipes are therefore unidirectional
- Require parent-child relationship between communicating processes



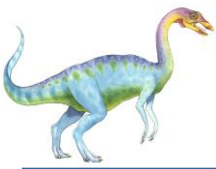
- Windows calls these **anonymous pipes**





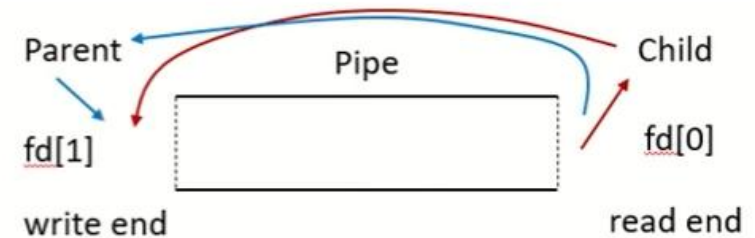
- This system call would create a pipe for one-way communication i.e., it creates two descriptors, first one is connected to read from the pipe and other one is connected to write into the pipe.
- A pipe file is created using the pipe system call
- Descriptor `pipedes[0]` is for reading and `pipedes[1]` is for writing. Whatever is written into `pipedes[1]` can be read from `pipedes[0]`.





■ Program to write and read message using pipe.

```
#define BUFFER_SIZE 25
int main(void) {
    char read_msg[BUFFER_SIZE];
    char write_msg[BUFFER_SIZE] = "Greetings";
    int fd[2];
    pid_t pid;
    if (pipe(fd) == -1)
    { fprintf(stderr, "Pipe failed"); return 1; }
    pid = fork();
    if (pid < 0) {
        fprintf(stderr, "Fork Failed"); return 1; }
    if (pid > 0) {
        close(fd[0]);
        fprintf(fd[1], write_msg, strlen(write_msg)+1);
        close(fd[1]); }
    ---
}
```



```
/* create pipe */

/* fork child process */

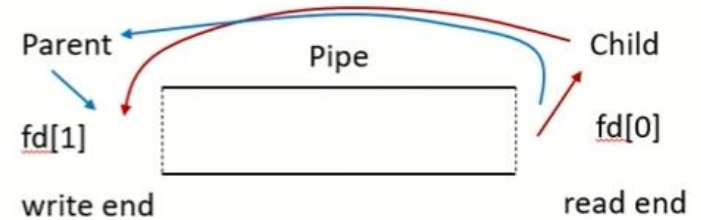
/* error */
/* parent process */
/* close unused end */
/* write to pipe */
/* close write end */
```





- Program to write and read messages using pipe.

```
---  
else {  
    close(fd[1]);  
    fscanf(fd[0], read_msg, BUFFER_SIZE);  
    printf("read %s", read_msg);  
    close(fd[0]); }  
return 0; }
```



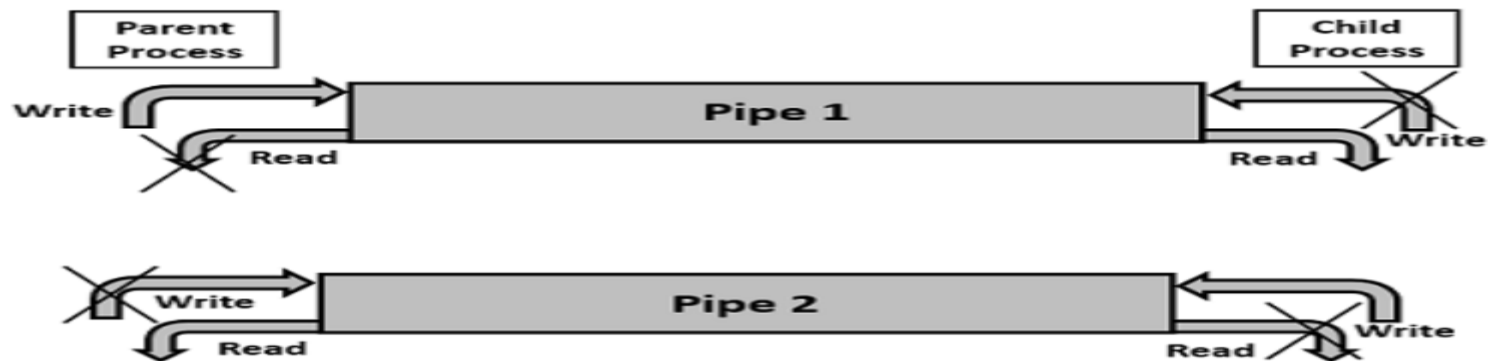
```
/* child process */  
/* close unused end */  
/* read from pipe */  
  
/* close read end */
```





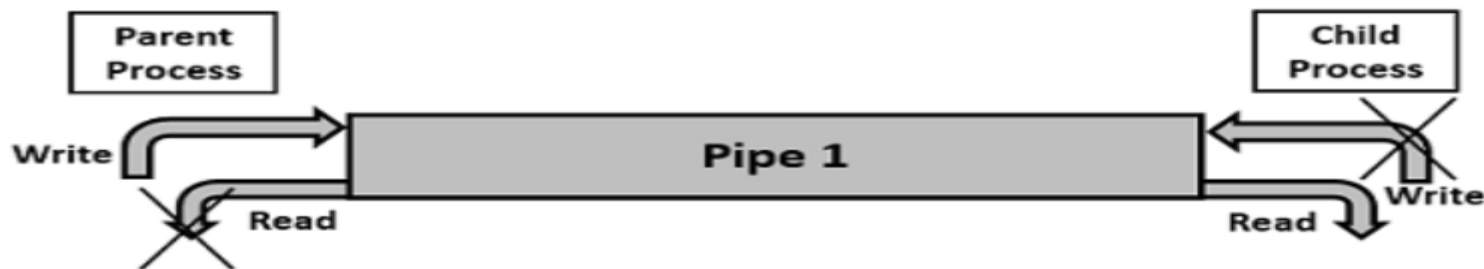
Two-way Communication Using Pipes

- Pipe communication is viewed as only one-way communication i.e., either the parent process writes and the child process reads or vice-versa but not both.
- However, what if both the parent and the child needs to write and read from the pipes simultaneously, the solution is a **two-way communication using pipes**. Two pipes are required to establish two-way communication.





- Suppose **two processes**, Process A and Process B, need to communicate. In such a case, it is important that the process which writes, closes its read end of the pipe and the process which reads, closes its write end of a pipe.
- Essentially, for a communication from Process A to Process B the following should happen.
- Process A should keep its write end open and close the read end of the pipe.
- Process B should keep its read end open and close its write end. When a pipe is created, it is given a fixed size in bytes.





Named Pipes

- Named Pipes are more powerful than ordinary pipes
- Communication is bidirectional
- No parent-child relationship is necessary between the communicating processes
- Several processes can use the named pipe for communication
- Provided on both UNIX and Windows systems
- A named pipe, also known as a FIFO (First-In, First-Out), is a special type of file that acts as a communication channel between unrelated processes.

```
mkfifo my_pipe
```

- This creates a named pipe file called `my_pipe` in the current directory.
- Any process can then open this file for reading or writing, and the data will be passed between the processes in a first-in, first-out manner.



End of Chapter 3

